

# TensorFlow, PyTorch and LiteRT

## THEORY SESSION 06



The frameworks, the model format and the runtime

24 September 2026 · Tiago Fonseca

# Frameworks, model format and runtime

## 01 TensorFlow and PyTorch

How a framework describes and learns a model.

## 02 LiteRT

Its origins, model representation and conversion tools.

## 03 The runtime

How a stored calculation becomes an executed calculation.

# TensorFlow



# TensorFlow

## 2015

**Google releases TensorFlow as open source.**

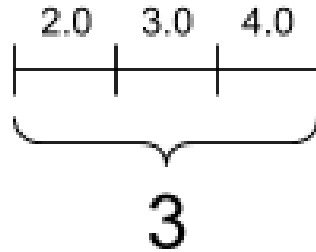
A framework for numerical computation and machine learning, with tools for building, training and exporting models.

# Tensors are the common language

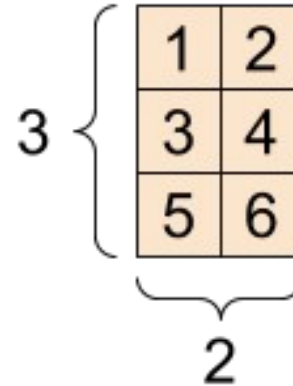
A tensor can be a single value or an array with one or more axes.

4

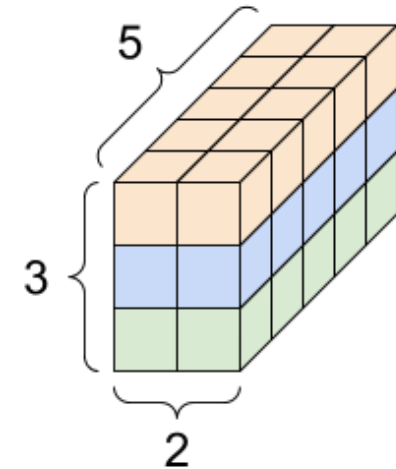
**Scalar**



**Vector**



**Matrix**



**Higher rank**

Frameworks use tensors for inputs, learned parameters and intermediate results.

Figures: TensorFlow, Introduction to Tensors.

# A value and its shape

## Scalar

**4**

**shape = ()**

Rank 0. One value.

## Vector

|   |   |   |   |
|---|---|---|---|
| 4 | 7 | 2 | 9 |
|---|---|---|---|

**shape = (4,)**

Rank 1. Four values.

Python shape notation: (4,) has one axis. (4) by itself is just the number 4.

# The same values, three different shapes

## Vector

**(3,)**

**[1, 2, 3]**

One axis

Rank 1

All three contain 3 values. An axis of length 1 still counts as an axis.

## Column matrix

**(3, 1)**

|   |
|---|
| 1 |
| 2 |
| 3 |

3 rows, 1 column

Rank 2

## Row matrix

**(1, 3)**

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
|---|---|---|

1 row, 3 columns

Rank 2

## Reading the shape (2, 3)

**3 columns**

|          |          |          |
|----------|----------|----------|
| <b>1</b> | <b>2</b> | <b>3</b> |
| <b>4</b> | <b>5</b> | <b>6</b> |

**shape = (2, 3)**

Axis 0: 2 rows

Axis 1: 3 columns

**Rank = 2**

**Size =  $2 \times 3 = 6$  values**

Shape lists axis lengths. Rank counts axes. Size counts individual values.

# Image shapes and the batch axis

| Example                  | Shape            | Meaning of the axes            |
|--------------------------|------------------|--------------------------------|
| One grayscale image      | (28, 28)         | height, width                  |
| An explicit channel axis | (28, 28, 1)      | height, width, channels        |
| A batch of 128 images    | (128, 28, 28, 1) | batch, height, width, channels |

**Here, channels come last. An RGB image has 3 channels.**

The last shape contains  $128 \times 28 \times 28 \times 1 = 100,352$  values and has rank 4.

# Tensor shapes through the PL2 model

| Stage               | Shape for a batch of 128 | What each example contains |
|---------------------|--------------------------|----------------------------|
| Input images        | (128, 28, 28)            | 28 × 28 pixel values       |
| Flatten             | (128, 784)               | 784 input values           |
| Dense(64) + ReLU    | (128, 64)                | 64 hidden activations      |
| Dense(10) + Softmax | (128, 10)                | 10 class scores            |

**Flatten preserves the batch axis. Each image gets its own 10 scores.**

In a model summary, None can mark an unspecified batch size, as in (None, 784).

## A Dense layer stores weights in a matrix

Each row of  $W$  holds one neuron's weights.

**W: shape (2, 3)**

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 0 | 2 | 1  |

×

**x: (3, 1)**

|   |
|---|
| 1 |
| 2 |
| 3 |

+

**b: (2, 1)**

|    |
|----|
| 1  |
| -2 |

**$z = Wx + b$  gives  $z$  with shape (2, 1)**

# One matrix product, two neurons

## Neuron 1

$$z_1 = 1 \times 1 + 0 \times 2 + (-1) \times 3 + 1 = -1$$

## Neuron 2

$$z_2 = 0 \times 1 + 2 \times 2 + 1 \times 3 - 2 = 5$$

**After ReLU:  $h_1 = 0$      $h_2 = 5$**

The matrix product computes both weighted sums. ReLU then acts on each result.

## Matrix calculations for a batch

Here, examples are rows. Keras stores weights as (inputs, units).

$$H = \text{ReLU}(XW_1 + b_1)$$

$$S = \text{Softmax}(HW_2 + b_2)$$

| Stage        | Input         | Weights           | Bias          | Result       |
|--------------|---------------|-------------------|---------------|--------------|
| Hidden layer | X: (128, 784) | $W_1$ : (784, 64) | $b_1$ : (64,) | H: (128, 64) |
| Output layer | H: (128, 64)  | $W_2$ : (64, 10)  | $b_2$ : (10,) | S: (128, 10) |

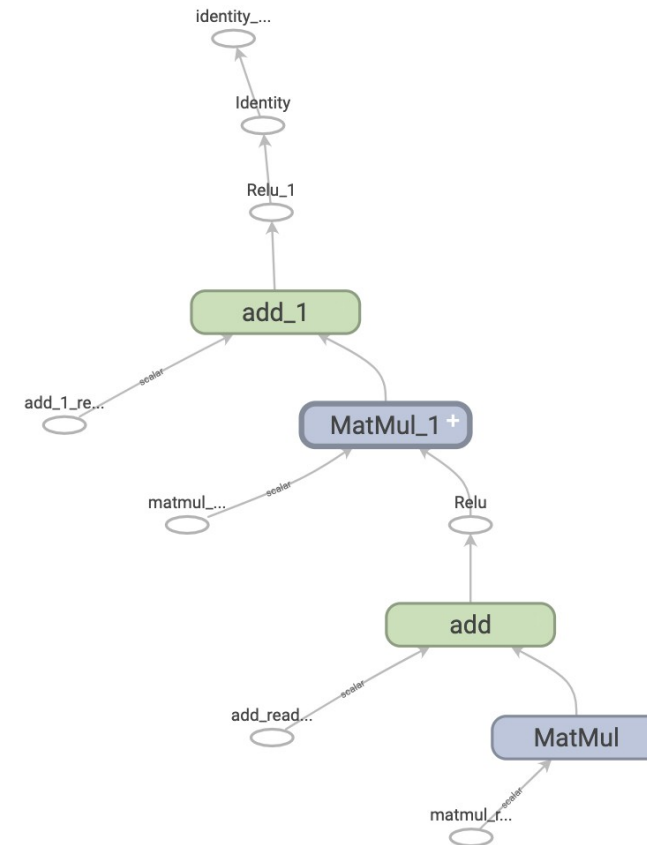
The same weights and biases are used for every image in the batch.

# A graph describes the calculation

**Operations are the nodes.**  
**Tensors connect them.**

The graph records which calculations depend on which values.

That structure can be saved and executed without the original Python program.



Official TensorBoard view of a two-layer TensorFlow network.

# Immediate execution and graph execution

## Eager execution

Operations run as the program reaches them.

The developer can inspect intermediate values during execution.

## Graph execution

The calculation is captured as a graph.

The system can analyze, transform and execute that representation.

**Modern TensorFlow supports both views of the same computation.**

# Where Keras fits

## A higher-level interface for model development.

Keras lets us describe models using layers and familiar training concepts.

In PL2, TensorFlow performs the underlying computation.



Keras 3 supports several backends. TensorFlow is the backend used in this course.

PyTorch



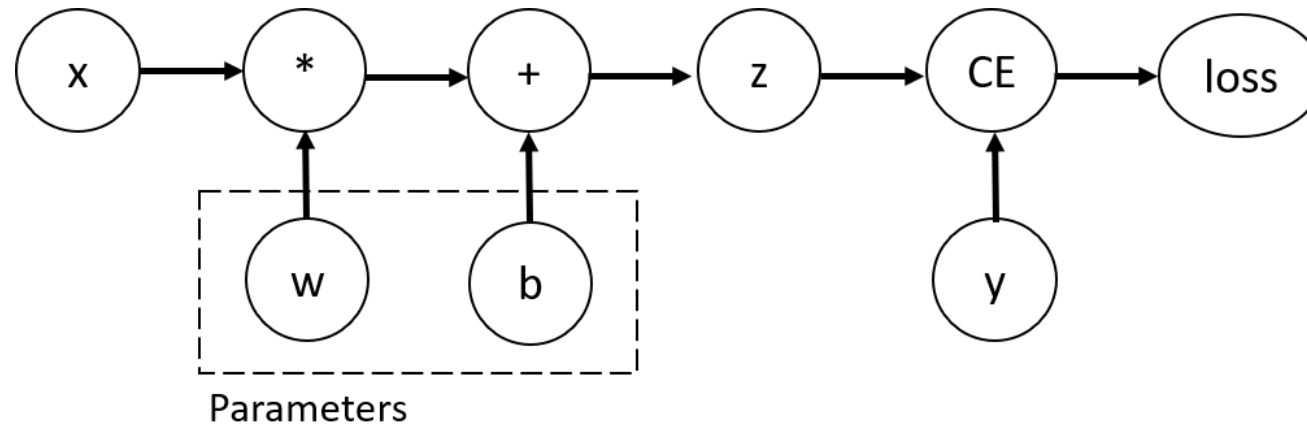
# PyTorch

**2017**

**The public PyTorch release.**

A framework built around tensor computation, Python model definitions and automatic differentiation.

## PyTorch records how a result was computed



**Autograd follows the operations used in the forward calculation.**

It uses those dependencies to compute gradients during training. The eager autograd graph is rebuilt as the program runs, which accommodates changing control flow.

Figure: PyTorch, Automatic Differentiation with torch.autograd.

# TensorFlow and PyTorch today

## TensorFlow

### Model building

Keras and lower-level APIs

### Gradients

Automatic differentiation

### Execution

Eager execution and graphs

### Beyond training

Multiple deployment routes

## PyTorch

Python modules and tensor operations

Autograd

Eager execution, compilation and export

Multiple deployment routes

**The concepts overlap. The interfaces, tools and export paths differ.**

## Why a separate inference runtime?

**The learned calculation can outlive the program that trained it.**

A training framework helps us construct and change a model.

An inference runtime executes a prepared representation of that model.

**LiteRT is one such execution environment.**

## LiteRT: Lite Runtime

# LiteRT

**Google's technology for  
on-device model execution.**

It brings together conversion tools, a model representation and runtime software.

The runtime reads a prepared model and performs the operations needed to compute its outputs.

# The TensorFlow Lite lineage

## 2017

### TensorFlow Lite

A dedicated path for running TensorFlow models on mobile and embedded systems.

## 2024

### LiteRT

The new name reflects a scope that extends beyond TensorFlow.

## Today

### A broader runtime

Multiple source frameworks and continued development of execution backends.

**The September 2024 rename retained the .tflite format and extension.**

## Several frameworks can feed the LiteRT ecosystem

**TensorFlow**

TensorFlow conversion

**PyTorch**

LiteRT Torch conversion

**JAX**

A supported JAX route

**A common destination: a model LiteRT can execute.**

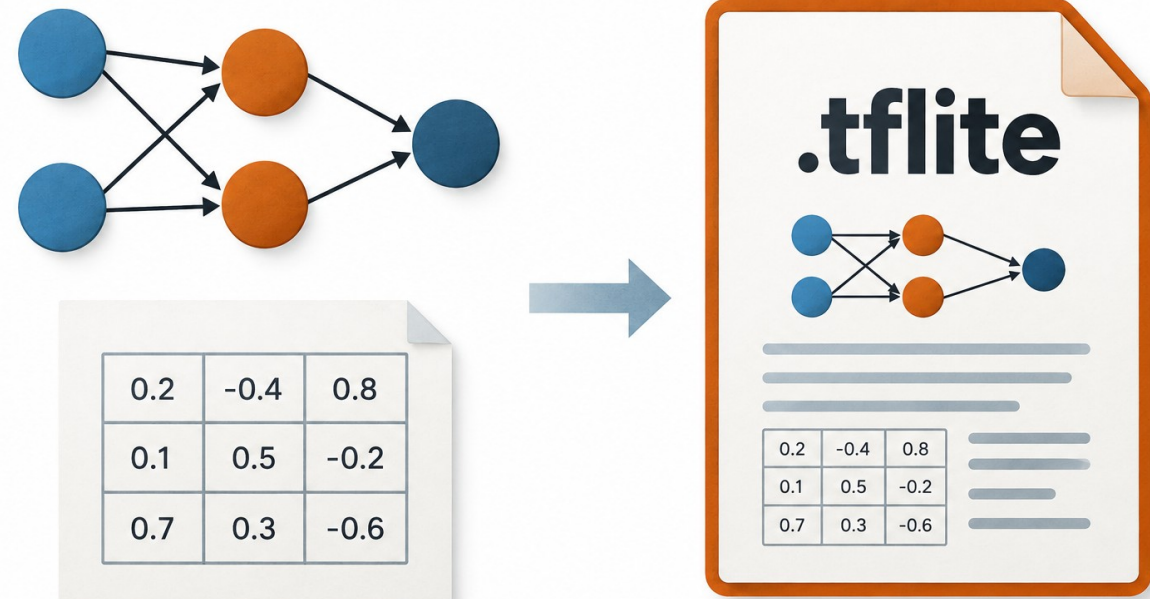
Each conversion route has its own requirements and supported operations.

# The .tflite file describes a computation

## Structure and learned values travel together.

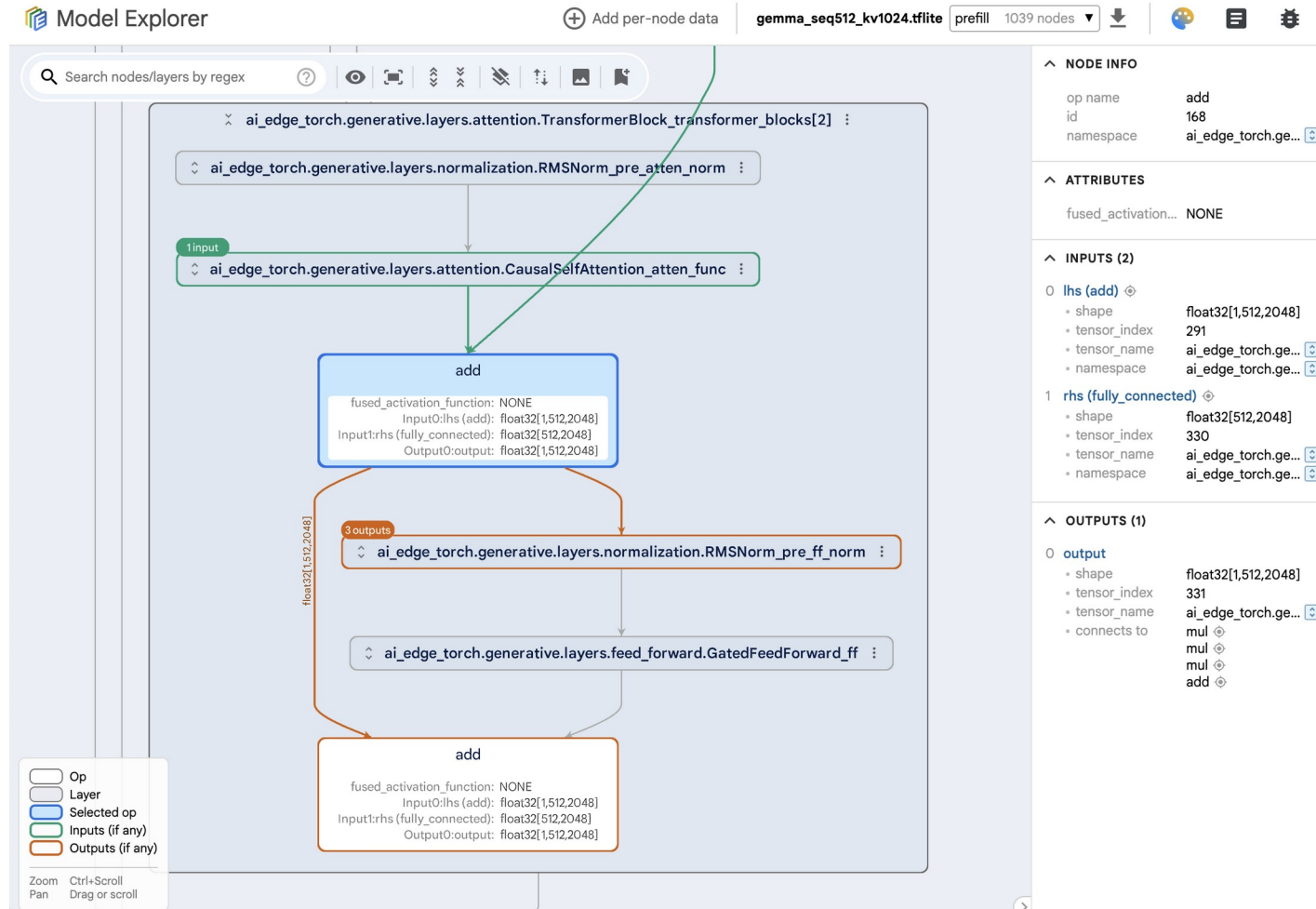
The file describes operations, their connections and the tensors they use.

It also stores constant data such as learned weights and biases.



Conceptual illustration. A model file still needs a compatible runtime to execute it.

# The graph remains visible inside a model file



Official Model Explorer example using Gemma. Read the structure, not every label.

## An operation

A node identifies a calculation.

## Its tensors

Inputs and outputs have explicit shapes and numeric types.

# Conversion is a translation of the model

## Source representation

The framework's description of the calculation and its parameters.

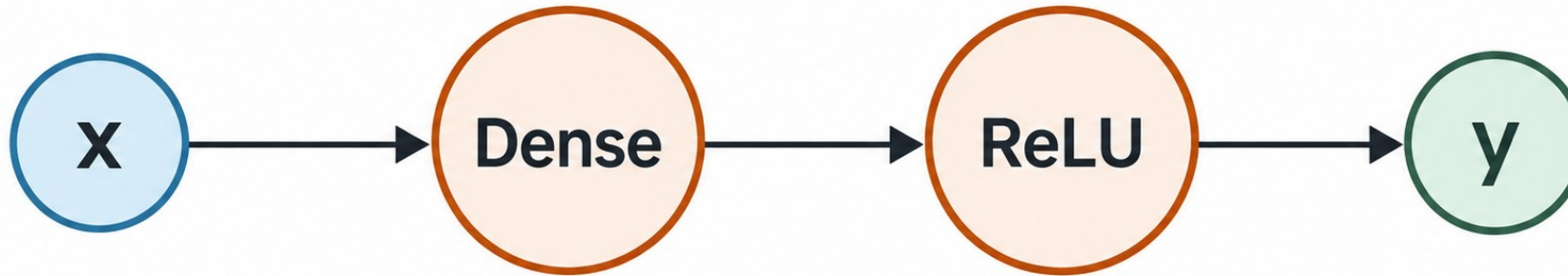
## LiteRT representation

A graph expressed using operations and data structures that LiteRT understands.

**The intended function stays the same. Its representation can change.**

## A graph can express the same work with fewer operations

### Separate operations

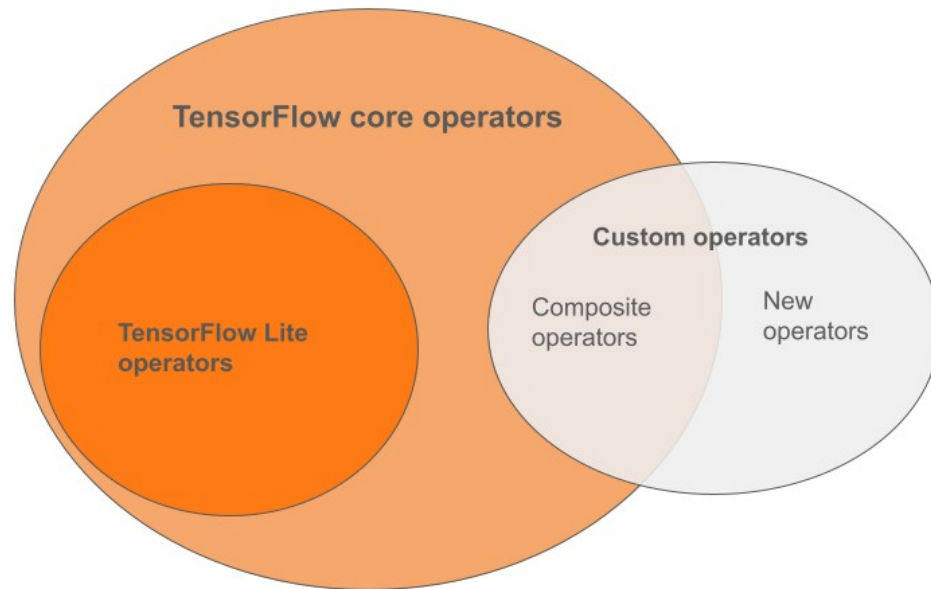


### Fused operation



Fusion combines supported operations while preserving the intended calculation.

# LiteRT has its own operator vocabulary



**Supported operations define what the runtime can express.**

Conversion maps the source calculation to that vocabulary.

An unsupported operation may require a different representation or additional runtime support.

Official TensorFlow conversion figure. It retains the historical “TensorFlow Lite” label.

# FlatBuffers is the storage format behind .tflite

## .tflite

## FlatBuffers

A structured binary format.

A schema defines how the model's information is organized.

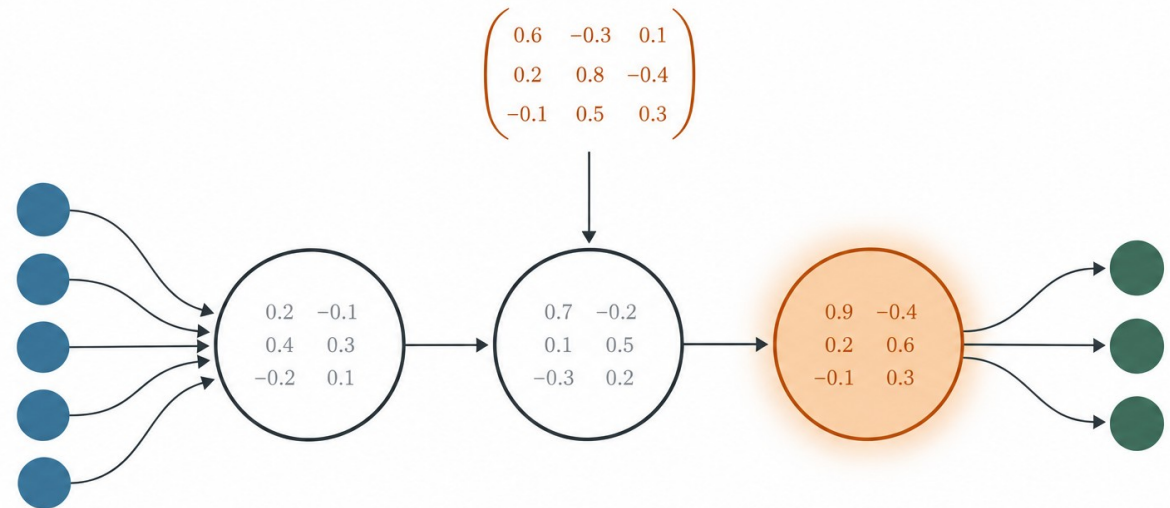
The runtime can access stored fields without first rebuilding the entire model as a separate object tree.

# The runtime turns the graph into execution

**A stored description becomes an ordered computation.**

LiteRT prepares the model's operations and tensor storage.

For each input, it executes the required work and makes the output tensors available.



Conceptual illustration. The highlighted node represents the operation currently executing.

# Operator, kernel and backend

## Operator

**The mathematical task.**

Example: a fully connected calculation.

## Kernel

**The implementation of that task.**

Code that performs the required numeric operations.

## Backend

**The execution path used for the work.**

For example, a supported CPU, GPU or NPU implementation.

# Two generations of runtime interface

## Interpreter

The established TensorFlow Lite interface.

The application prepares tensors and asks the runtime to execute the model.

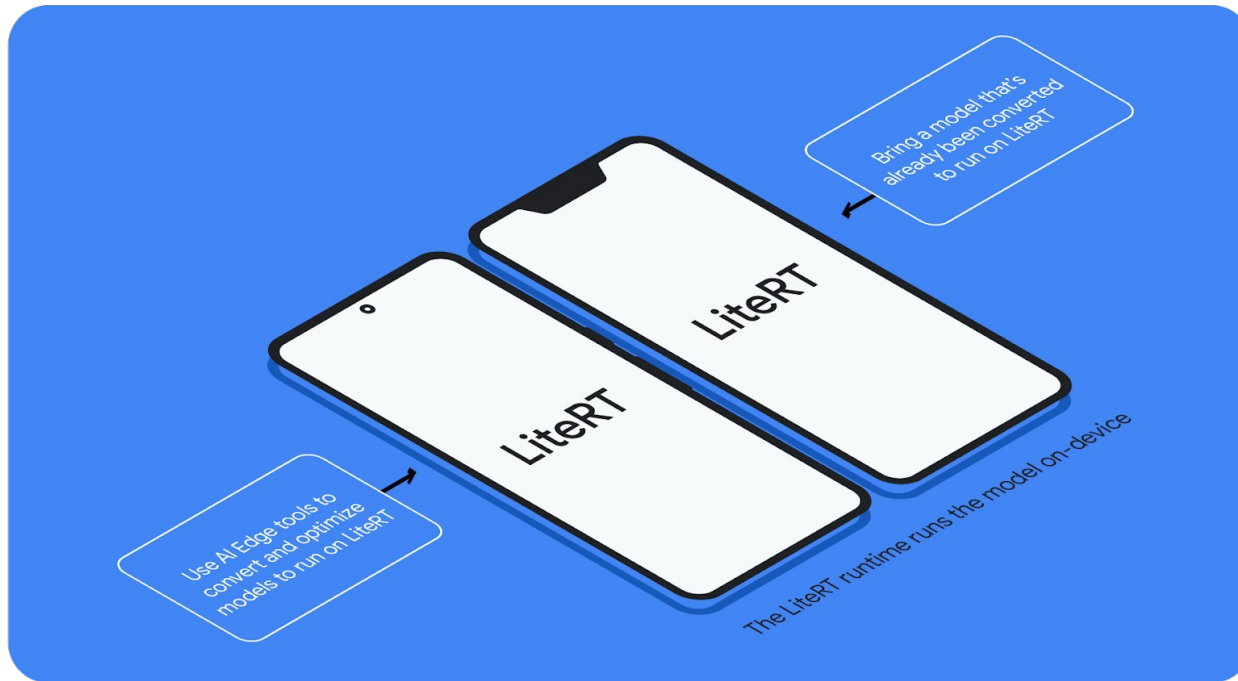
## CompiledModel

The modern LiteRT 2.x interface.

It brings target preparation, hardware acceleration and buffer handling into a newer execution model.

**Interpreter remains available. Google recommends CompiledModel for new LiteRT work.**

# LiteRT runs as software inside the application



**The runtime is a library the application uses.**

A model file describes the learned calculation.

The LiteRT library supplies the execution machinery that turns an input into an output.

Official illustration from the 2024 LiteRT announcement.

# Conversion and quantization describe different changes

## Conversion

Changes how the model is represented for a runtime.

## Quantization

Changes the numeric representation of some model values.

**A .tflite model can still contain Float32 weights and activations.**

# The LiteRT family has different scopes

## LiteRT

General runtime and conversion tools for supported models.

## Microcontrollers

A specialized execution path for microcontroller firmware.

## LiteRT-LM

Tools for running language models on the device.

**This lecture focuses on the conventional .tflite model and its runtime.**

**The roles are now distinct**

## **TensorFlow / PyTorch**

Express and learn the model.

## **Conversion**

Translate the learned calculation into a supported representation.

## **LiteRT runtime**

Execute that representation and produce the model's outputs.

# Further reading: the frameworks

## [TensorFlow basics](#)

Tensors, variables, automatic differentiation and model development.

## [Introduction to graphs](#)

How TensorFlow represents and executes a computation graph.

## [PyTorch: Learn the Basics](#)

The main abstractions in the PyTorch learning workflow.

## [Automatic differentiation in PyTorch](#)

How autograd records computations and obtains gradients.

# Further reading: LiteRT

## [TensorFlow Lite is now LiteRT](#)

The 2024 announcement and the continuity of the model format.

## [LiteRT overview](#)

Current framework support, runtime interfaces and scope.

## [Model conversion](#)

How a source model becomes a LiteRT representation.

## [On-device inference](#)

The current LiteRT execution model and runtime concepts.